🐧 Alpine VPS Complete Setup Script

Understanding Server Configuration for CS Students

Learning Objectives:

- Understand Linux server setup automation
- Learn security best practices
- Explore shell scripting techniques
- Practice server administration concepts

📋 Script Overview: What We're Building

```
#!/bin/bash
set -e  # Exit on any error
```

Key Features:

- **Automated deployment user creation**
- **Security hardening** (firewall, SSH, fail2ban)
- **Docker containerization platform**
- **Development environment setup**
- **Monitoring and logging systems**

Why Alpine Linux? Minimal, secure, perfect for learning server concepts!

🎯 Step 1: Error Handling & Logging

Production-Ready Error Handling:

```
set -e  # Exit on any error

error_exit() {
    echo -e "${RED}ERROR: $1${NC}" >&2
    exit 1
}

check_password() {
    if [ -z "$PASSWORD" ]; then
        error_exit "PASSWORD must be set"
    fi
}
```

Why This Matters:

- **Fail-fast principle**: Stop on first error
- **Clear error messages** for debugging
- **Logging everything** for audit trails

Real-world Application:

```
LOG_DIR="/var/log/deploy-setup"
exec > >(tee -a $LOG_DIR/setup.log)
```

🔐 Step 2: Password Security Implementation

Password Validation:

```
check_password() {
    if [ -z "$PASSWORD" ]; then
        error_exit "PASSWORD required"
    fi

    if [ ${#PASSWORD} -lt 8 ]; then
        error_exit "Password too short"
    fi
    success_msg "Password validated"
}
```

User Creation Process:

```
adduser -D deploy
echo "deploy:$PASSWORD" | chpasswd
```

Security Concepts:

- **Environment variables** for sensitive data
- **Input validation** before processing
- **Principle of least privilege**

Exercise: How would you modify this to require special characters in passwords?

🛡️ Step 3: Privilege Escalation (Sudo Setup)

Adding Sudo Capabilities:

```
apk add sudo
addgroup deploy wheel

# Enable wheel group in sudoers
sed -i 's/^# %wheel ALL=(ALL) ALL/
        %wheel ALL=(ALL) ALL/'
        /etc/sudoers
```

Why the `wheel` Group?

- **Historical convention** in Unix systems
- **Centralized privilege management**
- **Easy to audit** who has admin access

Testing Sudo Access:

```
sudo -u deploy sudo -n true 2>/dev/null
```

Security Note: Always backup `/etc/sudoers` before modification!

🔥 Step 4: Firewall Configuration (UFW)

UFW Setup Process:

```
apk add ufw
ufw --force reset
ufw default deny incoming
ufw default allow outgoing

# Allow specific services
for port in 22 80 443 9000 3000 8080; do
    ufw allow $port
done

ufw --force enable
```

Port Purpose Breakdown:

- **22** - SSH (secure shell)
- **80** - HTTP web traffic
- **443** - HTTPS encrypted web
- **9000** - Development server
- **3000** - Node.js common port
- **8080** - Alternative HTTP

Network Security Principle:

Default deny, explicitly allow

🔒 Step 5: SSH Hardening

Disabling Root Login:

```
cp /etc/ssh/sshd_config /etc/ssh/sshd_config.backup

sed -i 's/#*PermitRootLogin.*/
        PermitRootLogin no/'
        /etc/ssh/sshd_config

service sshd restart
```

Additional SSH Security:

```
# Optional: Disable password auth
# sed -i 's/#*PasswordAuthentication.*/
#         PasswordAuthentication no/'
#         /etc/ssh/sshd_config
```

Security Improvement:

- **Eliminate root attack vector**
- **Force use of dedicated user accounts**
- **Easier to track user activities**

Pro Tip: Always test SSH with new user before disconnecting!

🐳 Step 6: Docker Installation & Configuration

Docker Setup:

```
apk add docker docker-cli docker-compose
rc-update add docker boot
service docker start

# Add user to docker group
addgroup deploy docker
```

Docker Verification:

```
if service docker status &>/dev/null; then
    success_msg "Docker running"
else
    error_exit "Docker failed"
fi
```

Why Docker?

- **Containerization** for applications
- **Consistent environments**
- **Easy deployment and scaling**

Test Command: `sudo -u deploy docker run hello-world`

💻 Step 7: Development Tools Installation

```
apk add \
    git \               # Version control
    curl wget \         # HTTP clients
    nano vim \          # Text editors
    htop \              # Process monitor
    build-base \        # Compilation tools (gcc, make, etc.)
    openssl \           # Cryptographic library
    ca-certificates \   # SSL certificates
    zip unzip tar \     # Archive tools
    netstat-nat         # Network diagnostics
```

Educational Value: Each tool serves a specific development need!

🛡️ Step 8: Intrusion Detection (Fail2ban)

Fail2ban Configuration:

```
apk add fail2ban

cat > /etc/fail2ban/jail.local << 'EOF'
[DEFAULT]
bantime = 3600       # 1 hour ban
findtime = 600       # 10 min window
maxretry = 3         # 3 attempts

[sshd]
enabled = true
port = ssh
logpath = /var/log/auth.log
maxretry = 3
EOF
```

How Fail2ban Works:

1. **Monitors log files** for failed attempts
2. **Counts failures** within time window
3. **Automatically bans** offending IPs
4. **Releases bans** after timeout

Real-world Impact:

- **Prevents brute force attacks**
- **Reduces server load** from attackers
- **Automatic threat response**

📊 Step 9: Monitoring Scripts Creation

Disk Usage Monitor:

```
cat > /home/deploy/disk-check.sh << 'EOF'
#!/bin/bash
THRESHOLD=80
USAGE=$(df / | tail -1 | awk '{print $5}' | sed 's/%//')

if [ $USAGE -gt $THRESHOLD ]; then
    echo "WARNING: Disk usage is at ${USAGE}%" | logger
    echo "WARNING: Disk usage is at ${USAGE}%"
fi
EOF
```

System Information Script:

```
cat > /home/deploy/system-info.sh << 'EOF'
#!/bin/bash
echo "=== System Information ==="
echo "Hostname: $(hostname)"
echo "Uptime: $(uptime)"
echo "Disk Usage:"
df -h
echo "Memory Usage:"
free -h
echo "Docker Status:"
docker --version 2>/dev/null || echo "Docker not running"
EOF
```

Learning: Monitoring is crucial for production systems!

⚙️ Step 10: Environment Configuration

Environment File Creation:

```
cat > /home/deploy/.env << EOF
# Environment Configuration
DEPLOY_USER=deploy
ALLOWED_PORTS=${ports[*]}
SETUP_DATE=$(date)
DOCKER_INSTALLED=true
EOF

chown deploy:deploy /home/deploy/.env
```

Configuration Management:

- **Centralized settings** in one file
- **Easy to modify** without code changes
- **Version controlled** environment configs
- **Different configs** for dev/staging/prod

Best Practice: Never hardcode configuration in scripts!

🎨 Color-Coded Output System

```
# Color codes for better user experience
RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
NC='\033[0m' # No Color

success_msg() {
    echo -e "${GREEN}✓ $1${NC}"
}

error_exit() {
    echo -e "${RED}ERROR: $1${NC}" >&2
    exit 1
}

warning_msg() {
    echo -e "${YELLOW}⚠ $1${NC}"
}
```

🔄 Script Execution Flow

```
main() {
    echo -e "${GREEN}Starting Alpine VPS setup...${NC}"

    update_system                # 1. Update packages
    create_deploy_user           # 2. Create secure user
    setup_sudo                   # 3. Grant admin privileges
    setup_firewall              # 4. Configure network security
    setup_ssh_security          # 5. Harden SSH access
    install_docker              # 6. Install containerization
    install_development_tools    # 7. Add dev environment
    setup_fail2ban              # 8. Enable intrusion detection
    create_monitoring_scripts    # 9. Add monitoring tools
    setup_environment_file       # 10. Create config files

    echo -e "${GREEN}=== Setup completed! ===${NC}"
}
```

🧪 Testing Your Setup

Pre-execution Setup:

```
# Set password first
export PASSWORD="YourStrongPassword123"

# Add bash (Alpine uses ash by default)
apk add bash

# Run the script
bash server-first-setup.sh
```

Post-Setup Verification:

```
# Test SSH login
ssh deploy@your-server-ip

# Test Docker
sudo docker run hello-world

# Check firewall
sudo ufw status

# Monitor disk
./disk-check.sh

# System overview
./system-info.sh
```

🔍 Advanced Security Considerations

What This Script Achieves:

- ✅ User privilege separation
- ✅ Network access control
- ✅ SSH attack prevention
- ✅ Automated threat detection
- ✅ System monitoring

Additional Hardening Ideas:

- SSH key-only authentication
- Two-factor authentication
- Regular security updates
- Log aggregation and analysis

🎓 Learning Outcomes

Shell Scripting Concepts:

- Error handling and validation
- Function organization and modularity
- Environment variable usage
- File manipulation and permissions

System Administration Skills:

- User and group management
- Service configuration and management
- Network security implementation
- Monitoring and logging setup

DevOps Practices:

🚀 Next Steps & Assignments

Beginner Exercises:

1. Modify the script to add a new user with different permissions
2. Add a function to install a specific programming language (Python, Node.js)
3. Create a backup script for important configuration files

Intermediate Challenges:

1. Implement SSL certificate automation with Let's Encrypt
2. Add log rotation and retention policies
3. Create a rollback mechanism for failed deployments

Advanced Projects:

1. Convert the script into an Ansible playbook

💡 Key Takeaways

Production-Ready Principles:

- **Always validate inputs** before processing
- **Log everything** for debugging and compliance
- **Implement security by default**, not as an afterthought
- **Make scripts idempotent** (safe to run multiple times)
- **Provide clear feedback** to users

Real-World Applications:

- **Cloud server provisioning**
- **CI/CD pipeline setup**
- **Development environment automation**
- **Infrastructure consistency across teams**

❓ Questions & Discussion

Think About:

1. How would you modify this script for a production environment with 100+ servers?
2. What additional security measures would you implement for a financial application?
3. How could you make this script more portable across different Linux distributions?

Experiment:

- Run the script in a virtual machine
- Try breaking individual functions and observe error handling
- Modify the monitoring scripts to send alerts via email

Let's discuss your experiences and challenges! 🚀

📚 Additional Resources

Documentation:

- Alpine Linux Documentation
- Docker Official Docs
- UFW Ubuntu Firewall

Security References:

- CIS Benchmarks
- NIST Cybersecurity Framework

Practice Platforms:

- DigitalOcean Tutorials
- Vultr VPS Hosting